Object Dimensions Detector using a Single Camera

Ishaan CP SE21UARI051
Harshdeep J. SE21UARI044

1 Problem Formulation & Approach Overview

The objective of this project is to estimate the real-world dimensions (width, height, depth) and distance of objects captured in standard 2D images using only a single camera, such as one found on a smartphone. This work explores the feasibility of achieving dimensional estimation from dual-view (front/side) images.

The core challenge lies in resolving the inherent scale ambiguity present in single 2D images. An object appears smaller when farther away, and without additional information, its true size and distance cannot be determined from its pixel dimensions alone. To overcome this, our approach utilizes:

- **Monocular Depth Estimation:** A pre-trained deep learning model predicts a relative depth map from the input image(s).
- **Reference Object Calibration:** The user identifies a reference object within the scene whose real-world dimension (e.g., width) is known. This provides the crucial link to establish absolute scale.
- **Camera Parameter Integration:** Information about the camera's intrinsic properties (focal length or field of view) is used alongside the depth map and reference object to perform metric calculations.
- **Multi-View Fusion (Optional):** For estimating all three dimensions (Width, Height, Depth), the system accepts two views (front and side) of the object, calculating width/height from the front and depth/height from the side view, then combining the results.

This project does not involve training a new model but focuses on applying and integrating existing technologies into a user-friendly tool built with Streamlit. User-uploaded images serve as the primary input data.

2 Methodology

The dimension estimation process follows a sequential pipeline implemented within a Streamlit application:

2.1 Input and Configuration

The user initiates the process by:

1. Uploading one (front view) or two (front and side view) images in standard formats (JPG, PNG).
2. Specifying the known real-world width of a reference object (e.g., an ID card, default: 85.6mm) that is present in the image(s).
3. Providing camera intrinsic parameters: either the focal length in pixels or the horizontal and vertical field of view (FoV) in degrees. Accurate parameters are critical for metric accuracy.
4. (Optional) Adjusting advanced settings for object detection sensitivity (depth gradient threshold, minimum object size).

2.2 Depth Estimation

Upon user request for each view, a pre-trained DPT model (specifically 'Intel/dpt-hybrid-midas', based on MiDaS v3 Hybrid [1]) is used to predict a relative depth map from the input image. This model, accessed via the Hugging Face Transformers library, leverages a Vision Transformer backbone for robust dense prediction.

$$D_{relative} = \mathrm{DPT}(I_{input}) \tag{1}$$

where I_{input} is the input image and $D_{relative}$ is the resulting relative depth map (higher values typically mean farther away, but without metric scale). The prediction is resized to match the original image dimensions using bilinear interpolation. PyTorch serves as the backend for model inference.

2.3 Scale Calibration via Reference Object

This is the crucial step for converting relative depth to metric measurements.

1. The user draws a rectangular bounding box (Region of Interest - ROI) around the reference object directly on the displayed image using the 'streamlit-drawable-canvas' component.
2. The application extracts the pixel coordinates of the ROI $(x_{ref,1}, y_{ref,1}), (x_{ref,2}, y_{ref,2})$ and calculates its width in pixels, $w_{ref,px} = x_{ref,2} - x_{ref,1}$.
3. The median relative depth value, $d_{ref,rel}$, is computed from the depth map pixels within the reference ROI. Median is used for robustness against outliers.
4. Using the known real-world width $W_{ref,known}$, the pixel width $w_{ref,px}$, the relative depth $d_{ref,rel}$, and the camera parameters (focal length f or HFOV θ_H), a scale factor S is calculated. This factor relates relative depth values to absolute metric distances (e.g., meters).

- **Using Focal Length (f):** Based on the pinhole camera model approximation ($W/D \approx w/f$), the absolute distance to the reference D_{ref} can be related to S and $d_{ref,rel}$ as $D_{ref} = S/d_{ref,rel}$. Combining these gives:

$$S \approx \frac{W_{ref,known} \times f \times d_{ref,rel}}{w_{ref,px}} \tag{2}$$

- **Using Field of View (θ_H, θ_V):** The total scene width visible at distance D_{ref} is $W_{scene} = 2D_{ref} \tan(\theta_H/2)$. The ratio of object width to scene width equals the ratio of pixel width to image width ($W_{ref,known}/W_{scene} = w_{ref,px}/W_{img,px}$). Substituting $D_{ref} = S/d_{ref,rel}$ allows solving for S:

$$S \approx \frac{W_{ref,known} \times W_{img,px} \times d_{ref,rel}}{w_{ref,px} \times 2\tan(\theta_H/2)} \tag{3}$$

5. The estimated absolute distance to the reference object is then $D_{ref} = S/d_{ref,rel}$.

2.4 Target Object Detection and Measurement

Once the scale factor S is determined for a view:

1. Potential target objects (distinct from the reference) are automatically detected based on analyzing the depth map. A multi-stage approach is used:
   - First, edges are detected using depth gradients (Sobel filter) and thresholding. Morphological operations (closing) are applied to connect edges, and contours are found. Bounding boxes around large enough contours form initial candidates.
   - If gradient detection fails, a fallback method thresholds the depth map to find regions close to the minimum depth. Connected components analysis (labeling and region properties from 'scikit-image') identifies potential foreground objects.
   - If both fail, a small box around the absolute closest point in the depth map is used as a last resort.
2. Detected object bounding boxes are filtered to exclude the reference object (based on Intersection over Union - IoU) and limited to a maximum number specified by the user.
3. For each valid target object i:
   - Its bounding box ($w_{i,px}, h_{i,px}$) and median relative depth $d_{i,rel}$ are determined.
   - Its absolute distance is calculated: $D_i = S/d_{i,rel}$.
   - Its real-world dimensions (W_i, H_i) are estimated using the pinhole/FoV formulas rearranged:

$$W_i \approx \frac{w_{i,px} \times D_i}{f} \quad \text{(or FoV equivalent)} \tag{4}$$

$$H_i \approx \frac{h_{i,px} \times D_i}{f} \quad \text{(or FoV equivalent)} \tag{5}$$

2.5 Two-View Dimension Combination

If both front and side views are processed successfully:

- The front view provides estimates for Width (W_f) and Height (H_f).
- The side view provides estimates for Depth (D_s, derived from the measured 'width' in the side view) and Height (H_s).
- The final dimensions are reported as:
  - Final Width = W_f
  - Final Depth = D_s
  - Final Height = $(H_f + H_s)/2$ (Averaged)
  - Final Distance = $D_{target,front}$ (Distance from front view)

This combination assumes the object's width in the side view corresponds to its depth relative to the front view, and that height is consistent.

3 Experimental Results

The performance is demonstrated through the visual output and the plausibility of the estimated dimensions.

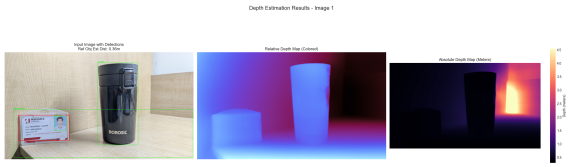
3.1 Setup and Workflow

A typical workflow involves:

1. Uploading an image (e.g., Figure 1a).
2. Running depth estimation (e.g., Figure 1b).
3. Selecting the reference object on the image via the interactive canvas (here, it is ID card).
4. Triggering the measurement process.
5. Detecting objects present in the image using image processing on the depth map.
6. Observing the results visualized on the image and displayed numerically (e.g., Figure 3a).
7. Repeating for the side view if 3D estimation is desired .
8. Reviewing the combined 3D results (Table 1).

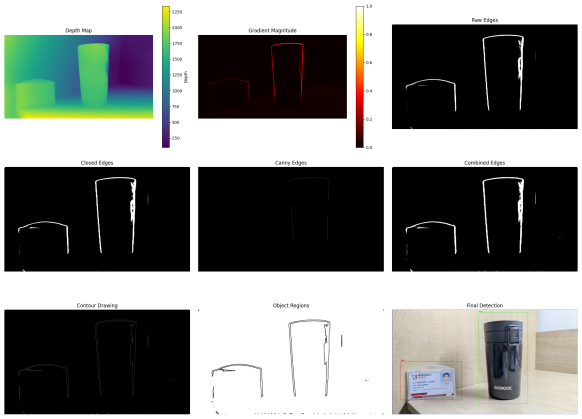


(a) Input Front View Image



(b) Estimated Depth Map

Figure 1: Input Image and Estimated Depth Map for the Front View.



(a) Object Detection

Figure 2: Detecting the object by applying image analysis on the depth map.



(a) Measurement Results Visualization (Front)

Figure 3: Reference Selection and Final Measurement Visualization for the Front View.

3.2 Example Combined Results

When both views are processed, the application provides combined 3D estimates. An example output might look like Table 1.

Table 1: Example Combined 3D Dimension Estimates

Dimension	Estimated Value
Width (from Front)	[8.5 cm]
Depth (from Side)	[8.5 cm]
Height (Avg)	[15.5 cm]
Distance (Front)	[0.20 m]

The accuracy of these results is highly dependent on the factors discussed below.

4 Discussion

This project demonstrates a practical application of monocular depth estimation for a common task. However, several factors influence its performance and reliability:

4.1 Strengths

- **Accessibility:** Utilizes standard camera images, making it widely applicable without specialized hardware.
- **Leverages Pre-trained Models:** Avoids the need for extensive data collection and model training by using powerful, readily available depth estimation models.

4.2 Limitations and Challenges

- **Depth Model Accuracy:** The entire process hinges on the quality of the depth map generated by the DPT model. These models can struggle with:
  - Transparent objects.
  - Complex scenes with significant occlusion.
- **Camera Parameter Sensitivity:** The accuracy of the provided focal length or FoV values is critical. Small errors in these parameters can lead to significant errors in metric estimations, especially for objects farther away. Obtaining accurate intrinsics for consumer cameras can be difficult.
- **Reference Object Dependency:** Requires a suitable reference object with a precisely known dimension to be clearly visible and correctly selected by the user. The assumption that the median depth within the ROI represents the object's distance might not hold for non-planar objects or oblique views.
- **Object Detection Robustness:** The depth-based object detection can be unreliable. It might merge adjacent objects at similar depths or fail to detect objects with subtle depth discontinuities.

- **Two-View Assumptions:** The combination logic for 3D dimensions assumes the object's orientation aligns perfectly with the front/side views and that the 'width' measured in the side view accurately represents the object's depth. This may not hold for complex shapes or misaligned views.

5 Conclusion

The "Object Dimensions Detector" project successfully implements a system for estimating real-world object sizes and distances from single or dual camera views using monocular depth estimation and user-provided calibration. It demonstrates the potential of combining deep learning models with geometric principles for practical measurement tasks accessible via standard cameras.

While the tool provides a functional demonstration, its accuracy is inherently limited by the performance of the underlying depth estimation model, the precision of the camera parameters supplied by the user, and the correct identification of the reference object. It serves as a valuable proof-of-concept but should be used with an understanding of its sensitivities and limitations. Future work could explore more robust object detection methods, ways to estimate camera parameters automatically, or integration with more advanced 3D reconstruction techniques.

References

[1] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer". In: **IEEE Transactions on Pattern Analysis and Machine Intelligence (PAMI)**. Vol. 44. 3. 2022, pp. 1623–1637. DOI: 10.1109/TPAMI.2020.3019967.